— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 6

9 Semantics of Programming Languages (nk480)

Subtyping

(*a*) Suppose the types $\mathsf{InputChannel}(\tau)$, $\mathsf{OutputChannel}(\tau)$, and $\mathsf{IOChannel}(\tau)$ have the following API:

$$\begin{array}{lcl} \mathsf{read} & : & \mathsf{InputChannel}(\tau) \to \tau \\ \\ \mathsf{write} & : & \mathsf{OutputChannel}(\tau) \times \tau \to \mathsf{unit} \\ \\ \mathsf{read} & : & \mathsf{IOChannel}(\tau) \to \tau \\ \mathsf{write} & : & \mathsf{IOChannel}(\tau) \times \tau \to \mathsf{unit} \end{array}$$

Note that read and write are overloaded functions. Define a suitable subtyping relation over the channel types. [5 marks]

Answer:

$$\frac{\tau \leq \tau'}{\mathsf{InputChannel}(\tau) \leq \mathsf{InputChannel}(\tau')} \qquad \frac{\tau' \leq \tau}{\mathsf{OutputChannel}(\tau) \leq \mathsf{OutputChannel}(\tau')}$$

$$\frac{\tau' \leq \tau \qquad \tau \leq \tau'}{\mathsf{IOChannel}(\tau) \leq \mathsf{IOChannel}(\tau')}$$

$$\frac{}{\mathsf{IOChannel}(\tau) \leq \mathsf{InputChannel}(\tau)} \qquad \frac{}{\mathsf{IOChannel}(\tau) \leq \mathsf{OutputChannel}(\tau)}$$

Concurrency

(*b*) Consider the following two concurrent L1 programs:

Program 1: $\mathsf{r} := !\mathsf{r} + 1; \mathsf{r} := !\mathsf{r} + 1$

Program 2: $\mathsf{r} := !\mathsf{r} + 2$

Are these two programs semantically equivalent in a concurrent setting? Give an informal but precise argument if they are, or give a counterexample if not. [2 marks]

Answer: They are NOT equivalent. If r starts out at 0, and the two programs are run in parallel with the $\mathsf{r} := 5$, then the program on the right can either return 5 or 7. However, the program on the left can return more values (such as 6).

State and Exceptions

(*c*) Suppose we try to introduce a safe file-handling API in a language with higher-order functions and state (such as L3) by introducing the function $\mathsf{withFile} : (\mathsf{File} \to \mathsf{unit}) \to \mathsf{unit}$. This function creates a file, passes the file to its callback, and then closes the file after the callback returns. Is there any way for a File object to outlive the callback invocation and leak into the environment? Either argue that the API is safe, or give a counterexample. [3 marks]

Answer: This API is not safe. If we have an exception which can carry a file (in Ocaml, `exception Foo of file`), we can simply write.

```
let file =
  try
    withFile (fun file -> raise (Foo file))
  with
   Foo file -> file
```

We can also leak files by writing to a shared reference cell:

```
let file =
  let r = ref (fun () -> assert false) in
  let () = withFile (fun file -> r := (fun () -> file)) in
  (!r)()
```

Inductive Proofs

(*d*) We can define the prefix relation $xs \sqsubseteq ys$ on lists as follows:

$$\frac{}{[] \sqsubseteq ys} \qquad \frac{xs \sqsubseteq ys}{x :: xs \sqsubseteq x :: ys}$$

(*i*) Prove that the prefix relation is reflexive. [3 marks]

Answer: We prove that $xs \sqsubseteq xs$ holds by structural induction on xs.

- Case $xs = []$.

  This case is immediate from the rules.

- Case $xs = x :: xs'$

  By induction, we have that $xs' \sqsubseteq xs'$.
  By rule, we have that $x :: xs' \sqsubseteq x :: xs'$.

(*ii*) Prove that the prefix relation is transitive. Inversion properties may be used without proof, as long as they are explicitly indicated. [7 marks]

Answer: We want to show that if $xs \sqsubseteq ys$ and $ys \sqsubseteq zs$, then $xs \sqsubseteq zs$. We can do this by structural induction on ys.

Assume we have $xs \sqsubseteq ys$ and $ys \sqsubseteq zs$.

- Case $ys = []$.

  In this case, we know by inversion on the prefix relation $xs \sqsubseteq []$ that $xs = []$.
  Since $xs = ys = []$, it follows that since we have $ys \sqsubseteq zs$, we have $xs \sqsubseteq zs$.

- Case $ys = y :: ys'$.

  In this case, inversion on $ys \sqsubseteq zs$ tells us that $zs = y :: zs'$ and $ys' \sqsubseteq zs'$.
  Inversion on $xs \sqsubseteq ys$ tells us that either $xs = []$ or $xs = y :: xs'$ and $ys' \sqsubseteq zs'$.
  If $xs = []$, then we have $xs \sqsubseteq zs$ immediately by rule.
  If $xs = y :: xs'$, then by induction on $xs' \sqsubseteq ys'$ and $ys' \sqsubseteq zs'$, we have that $xs' \sqsubseteq zs'$.
  Then, by rule we have $y :: xs' \sqsubseteq y :: zs'$.
  This is the same as $xs \sqsubseteq zs$.
